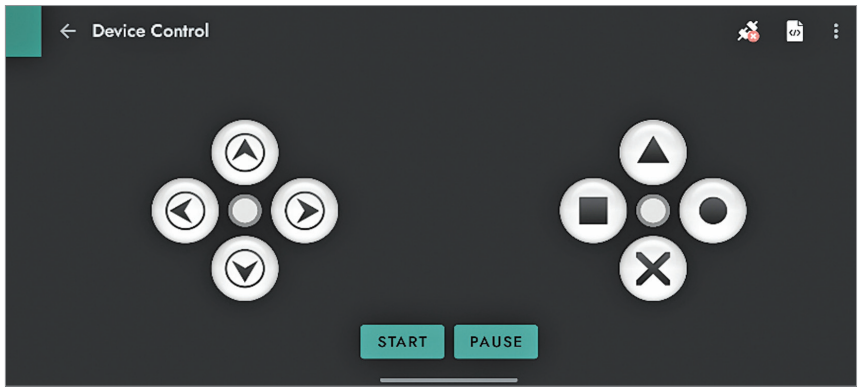


Fig. 4: Control panel for robot control

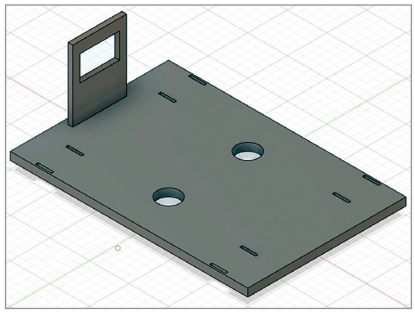


Fig. 5: 3D printed robot chassis

motor-control logic, and OLED display updates. Since the microcontroller cannot drive the motors directly, an L298N H-bridge motor driver controls them. This arrangement allows the robot to move forwards, backwards, left, right, and rotate through 360 degrees.

The L298N's IN1, IN2, IN3, and IN4 pins are connected to pins A7, A6, A5, and A4 of the STM32 Blue Pill, respectively. These signals control the direction of the motors connected to the driver. IN1 and IN2 control motor A, while IN3 and IN4 control motor B. By setting these GPIO pins high or low, the controller determines the direction of rotation or stops the motors. For example, when IN1 is high and IN2 is low, motor A rotates in one direction; reversing these levels changes its direction. The same logic applies to IN3 and IN4 for motor B. By combining the directions of both motors, the robot can move forwards, backwards, left, or right.

The HC-05 Bluetooth module provides wireless control of the robot through UART serial communication. Its TX and RX pins are connected to the corresponding RX and TX pins of the STM32 in a crossover configuration. The TX pin of the HC-05 is connected to the RX pin of the STM32, while its RX pin is connected to the TX pin. This allows commands received wirelessly to reach the controller.

A compact SSD1306 OLED display provides real-time feedback, including animation and the robot's movement direction. It communicates with the STM32 through the I2C interface. The VCC pin is connected to the STM32's 3.3V supply, GND to common ground, SDA to PB7, and SCL to PB6.

The robot is powered by two 3.7V, 2000mAh-2500mAh Li-ion batteries connected through a two-cell holder, providing a nominal 7.4V supply to the L298N motor driver. The STM32 Blue Pill is supplied with 5V through the motor driver's 5V output.

Software

This system requires four files to operate the OLED display. The ssd1306.h header file contains the display settings, including the width (128 pixels), height (64 pixels), I2C address (0x78), colour definitions, and function declarations. The ssd1306.c file contains the functions required to operate the display, including initialising the display, filling the screen, drawing individual pixels, writing characters and strings, and updating the screen buffer. The fonts.h header file defines the available fonts: Font_7x10, Font_11x18, and Font_16x26. The fonts.c file contains the pixel data for these fonts, which is used to display characters on the OLED. The ssd1306.c and fonts.c files work together to drive the OLED display.

The robot is operated using the Arduino Bluetooth controller mobile application, which is available on the Google Play Store. The application sends commands via the HC-05 Bluetooth module to the STM32 Blue Pill. It provides buttons for controlling the robot's movement. Pressing the up arrow moves the robot forwards, the down arrow moves it backwards, the left arrow turns it left, and the right arrow turns it right. Other buttons perform specific functions: the square button changes the OLED animation, the circle button makes the robot rotate 360 degrees, and the cross button immediately stops the robot by deactivating the motors. The start button activates Bluetooth communication and begins sending commands, while the pause button stops command transmission.

All commands are transmitted through the UART1 interface as single-character signals to pin PA10 of the STM32 Blue Pill. The received commands control the GPIO pins PA4 (left-front motor), PA5 (left-rear motor), PA6 (right-front motor), and PA7 (right-rear motor). Fig. 4 shows the robot control panel.

STM32CubeIDE software is used to program the STM32 Blue Pill. The following steps describe how to program the board.

Step 1: Create the system. Open STM32CubeIDE, select the STM32F103C8T6 microcontroller, and click Next. Enter the system name as bluetooth_car and click Finish. When prompted to initialise all peripherals, click Yes.

Step 2: Configure UART1. In the configuration panel, go to Connectivity, select UART1, and set the mode to asynchronous. This automatically